

Project Reference Documentation

1. Application Overview

This full-stack application provides an insurance platform with features for policy management, claim processing, notifications, invoices, and user authentication.

- **Frontend:** Angular
- **Backend:** .NET 9 Web API
- **External Services:** Vercel Blob (for secure file storage, e.g., documents/invoices), Quest PDF (for generating PDF receipts and policy documents).

System Roles

- **Customer:** Browses plans, requests policies, pays premiums, submits claims, views invoices.
 - **Agent:** Reviews and approves policy requests based on risk score and documents.
 - **ClaimOfficer:** Reviews, approves, or rejects submitted claims and processes payouts.
 - **Admin:** Manages plans, users, and oversees all system data.
-

2. Backend Entities (Data Schema)

User

Represents a user in the system (Customer, Agent, ClaimOfficer, or Admin).

- `Id` (Guid): Unique identifier.
- `FirstName` , `LastName` , `Email` (string): User details.
- `PasswordHash` (string): Encrypted password.
- `Role` (UserRole): Role type.
- `GovernmentId` , `Address` , `Phone` , `DateOfBirth` (string/DateTime): Optional details.
- `IsDeleted` (bool): Soft delete flag.
- **Relationships:** Policies, Claims, Invoices, Notifications.

Plan

Insurance plan templates that customers can subscribe to.

- `Id` (Guid): Unique identifier.
- `Name` , `Description` (string): Plan details.
- `PremiumAmount` , `CoverageAmount` (decimal): Base pricing and coverage.
- `DurationInMonths` (int): Standard term.
- `PaymentFrequency` , `PlanType` (string): Terms of payment and type of plan.
- `IsActive` (bool): Visibility flag.
- **Relationships:** Policies.

Policy

An active insurance policy belonging to a user.

- `Id` , `UserId` , `PlanId` , `AgentId` (Guid): Identifiers.
- `StartDate` , `EndDate` (DateTime): Term of the policy.
- `DurationInMonths` (int): Chosen term length.
- `PaymentFrequency` (PaymentFrequency): Chosen payment frequency.

- `Status` (`PolicyStatus`): Active, Expired, Cancelled, etc.
- `TotalPremium` , `TotalPaid` (decimal): Payment tracking.
- `PlanBaseCoverageAmount` , `PlanBasePremiumAmount` (decimal): Snapshots of plan values at creation.
- `CoverageAmount` (decimal): Calculated coverage specific to this policy.
- **Relationships:** Payments, Claims.

PolicyPayment

Premium payments collected against an active policy.

- `Id` , `PolicyId` (Guid): Identifiers.
- `Amount` (decimal): Payment amount.
- `DueDate` , `PaidDate` (DateTime): Chronology.
- `Status` (`PaymentStatus`): Pending, Paid, Overdue.

PolicyRequest

Requests by users to buy an insurance plan, pending Agent approval.

- `Id` , `UserId` , `PlanId` , `AgentId` (Guid): Identifiers.
- `DurationInMonths` (int): Requested term.
- `PaymentFrequency` (`PaymentFrequency`): Requested frequency.
- `RiskScore` (decimal): Computed systemic risk.
- `PanDocumentUrl` , `AddressProofUrl` (string): URLs to documents (stored in Vercel Blob).
- `Status` (`PolicyRequestStatus`): Pending, Approved, Rejected.
- `RejectionReason` (string): populated if rejected.

Claim

Insurance claims submitted by users against a policy.

- `Id` , `PolicyId` , `UserId` , `ClaimOfficerId` (Guid): Identifiers.
- `Reason` (string): User submitted reason.
- `DocumentUrl` , `DocumentHash` (string): Evidence material.
- `ClaimAmount` , `ApprovedAmount` (decimal): Requested and finalized values.
- `Status` (`ClaimStatus`): Pending, Approved, Rejected.
- `SubmittedAt` , `ProcessedAt` (DateTime): Chronology.
- **Relationships:** ClaimPayment.

ClaimPayment

Disbursement of funds for an approved claim.

- `Id` , `ClaimId` (Guid): Identifiers.
- `AmountPaid` (decimal): Payout value.
- `PaidAt` (DateTime): Payout timestamp.

Invoice

Auto-generated receipts/documents using Quest PDF.

- `Id` , `UserId` , `ReferenceId` (Guid): Identifiers.
- `Type` (`InvoiceType`): Type of invoice (Policy, Claim, Payment).
- `FileUrl` (string): Link to the generated PDF (Vercel Blob).

Notification

System alerts to users.

- `Id` (int): Auto-incremented ID.
- `UserId` (Guid): Target user.
- `Title` , `Message` (string): Alert content.
- `IsRead` (bool): Read status.

3. API Endpoints

3.1 Authentication

- `POST /api/auth/register` (Public)
 - Registers a new user. Initializes role as `Customer` .
- `POST /api/auth/login` (Public)
 - Authenticates credentials and returns a JWT token representing the user's role and identity.

3.2 Users

- `GET /api/users/me` (Any Authenticated User)
 - Retrieves profile information for the currently logged-in user.
- `PUT /api/users/me` (Any Authenticated User)
 - Modifies the user's profile details natively.
- `GET /api/users` (Admin)
 - Returns a system-wide user directory for management.
- `PUT /api/users/{id}/role` (Admin)
 - Upgrades or downgrades a user's role (e.g., Customer to Agent).
- `DELETE /api/users/{id}` (Admin)
 - Soft-deletes a user from the system.

3.3 Plans

- `GET /api/plans` (Customer, Admin)
 - Lists all active insurance plans available for purchase.
- `GET /api/plans/{id:guid}` (Customer, Admin)
 - Retrieves full details for a specific plan.
- `POST /api/plans` (Admin)
 - Creates a new insurance plan template to offer to customers.
- `PUT /api/plans/{id:guid}` (Admin)
 - Adjusts details or pricing of a plan.
- `DELETE /api/plans/{id:guid}` (Admin)
 - Removes or deactivates a plan so no further policies can use it.

3.4 Policy Requests

- `POST /api/policy-requests` (Customer)
 - Initiates the policy-buying process. Requires multi-part form data including PAN and Address Proof files which are uploaded to Vercel Blob.
- `GET /api/policy-requests/my` (Customer)
 - Displays the logged-in customer's policy requests along with their current status.
- `GET /api/policy-requests/assigned` (Agent)

- Displays pending system-wide policy requests that agents need to verify and process.
- **POST /api/policy-requests/{id:guid}/approve** (Agent)
 - Approves a customer's request. This sets the stage for the customer to finalize the purchase.
- **POST /api/policy-requests/{id:guid}/reject** (Agent)
 - Rejects a request with an explicit reason.

3.5 Policies

- **POST /api/policies/purchase/{requestId:guid}** (Customer)
 - Finalizes the purchase of an approved policy request, converting it into an active `Policy`. Triggers generation of initial payment schedules/invoices.
- **GET /api/policies/my** (Customer)
 - Returns all active and historic policies belonging to the current user.
- **GET /api/policies/{policyId:guid}** (Any Authenticated User)
 - Fetches specific policy data. Access is isolated so customers only view their own policies.
- **POST /api/policies/pay/{paymentId:guid}** (Customer)
 - Completes payment against a pending policy premium schedule. Utilizes Quest PDF to natively generate an invoice pushed to Vercel Blob.
- **GET /api/policies/assigned** (Agent)
 - Views all active policies brokered by the authenticated agent.
- **GET /api/policies/all** (Admin)
 - Returns all policies system-wide for overarching review.

3.6 Claims

- **POST /api/claims** (Customer)
 - Files a claim against an active policy. May upload evidential documents which are processed and stored transparently.
- **GET /api/claims/my** (Customer)
 - Returns the lifecycle status for claims formulated by the user.
- **GET /api/claims** (Admin)
 - Views all claims in the platform.
- **GET /api/claims/assigned** (ClaimOfficer)
 - Obtains the queue of claims waiting to be audited by the logged-in Claim Officer.
- **POST /api/claims/{claimId:guid}/approve** (ClaimOfficer)
 - Approves a vetted claim for a configured sum. Spawns an internal `ClaimPayment` and generates a settlement invoice.
- **POST /api/claims/{claimId:guid}/reject** (ClaimOfficer)
 - Marks an invalid claim as rejected.

3.7 Invoices

- **GET /api/invoices/my** (Customer)
 - Consolidates all generated Quest PDF receipts (premium payments and claim payouts) and enables the user to download them via stored Vercel Blob URLs.

3.8 Notifications

- **GET /api/notifications** (Any Authenticated User)

- Retrieves the latest 50 notifications alerting the user to policy or claim state changes.
- **GET /api/notifications/unread-count** (Any Authenticated User)
 - Useful for polling to show badge indicators in frontend interfaces.
- **PUT /api/notifications/{id}/mark-read** (Any Authenticated User)
 - Acknowledges a single notification.
- **PUT /api/notifications/mark-all-read** (Any Authenticated User)
 - Broadly marks all user notifications as processed.

4. Core Business Flows

4.1 Policy Request & Buying Flow

1. **Initiation:** A customer submits a `PolicyRequest` for a chosen plan via a multi-part form payload. This includes their chosen term duration in months, payment frequency, and KYC documents (PAN and Address Proof).
2. **Document Processing:** The backend captures the byte streams of the documents, computes a SHA-256 integrity hash, and securely uploads them directly to Vercel Blob Storage, storing the URL pointers on the database request.
3. **Risk & Assignment:** The system leverages a baseline logic to calculate a `RiskScore` combining a base volatility metric and the policy length. The request is subsequently assigned randomly to an active Agent to review.
4. **Agent Underwriting:** The assigned Agent sees the request in their portal, downloads/views the KYC documents natively via Vercel URLs, considers the risk score, and either `Approves` or `Rejects` the request.
5. **Execution (Purchase):** If approved, the Customer acts to purchase the policy.
 - The system snapshots the Plan's current `BaseCoverageAmount` and `BasePremiumAmount` natively to the Policy so later plan updates do not retroactively alter the contract.
 - It calculates the `CoverageAmount` proportionally based on the duration ratio against the plan defaults.
 - An array of `PolicyPayment` entities are spawned sequentially mapping the full scheduled interval (Monthly, Quarterly, Yearly).
 - The system automatically marks the very first premium payment as `Paid`, registers the incoming funds, and kicks off async background workflows to generate the Policy Invoice and Payment Invoice utilizing Quest PDF. Both PDFs are uploaded to Vercel.

4.2 Claim Processing Flow

1. **Submission:** A Customer files a `Claim` outlining the details of their distress, uploading an evidentiary document and the monetary amount requested.
2. **Automated Vetting:** The system audits the claim amount against the `CoverageAmount` limits of the snapshot Policy instance. If within bounds, it employs a Round-Robin style distribution algorithm looking for the `ClaimOfficer` with the lowest queue of pending responsibilities, and assigns the claim to them.
3. **Investigation:** The Claim Officer assesses the provided reason, the snapshot history of payments against the policy, and evaluates the evidence.
4. **Settlement Mechanism:**
 - *On Approval:* The Officer decides on the `ApprovedAmount`. The system flips the state to `Approved`, records a completed `ClaimPayment` transaction matching the approved funds, generating an official Settlement Invoice (via Quest PDF), and alerts the customer.
 - *On Rejection:* The system flips the state to `Rejected`, halts any fiscal disbursements, generates a formal Rejection Invoice for legal consistency, and notifies the customer.

5. Third-Party Integrations

- **Quest PDF:** Leveraged internally by the `InvoiceGeneratorService`. Utilizing the Fluent API, the C# application spins up native A4-sized PDF files featuring the corporate logo, structured table generation, dynamic data binding mapping policy history or active claims, scaling seamlessly across multiple pages. These are issued predominantly as Policy Confirmation deeds, Financial Payment Receipts, and Claim Statements.
- **Vercel Blob Storage:** Leveraged via the native `HttpClient` wrapper mimicking an SDK wrapper. Generates explicit `application/pdf` streams directly onto remote clusters via an authorization bearer JWT. This resolves infrastructure constraints around persistent storage states while securely delivering unique access URLs retained in PostgreSQL/SQLServer `Invoice` items.

6. Frontend Architecture & logic

The frontend is a Single Page Application built using **Angular** (TypeScript). It employs modern state management using Angular **Signals** and orchestrates HTTP communications securely using Interceptors.

6.1 Frontend Application Flow

1. **Authentication & State Bootstrapping:** When a user arrives, `AuthService` reads `sessionStorage` to boot up the persistent JWT token. If valid, the JWT is decoded via `jwt-decode` to extract the exact role (`http://schemas.microsoft.com/ws/2008/06/identity/claims/role`). This is broadcasted reactively via a Signal (`currentUser`).
2. **Access Control:** Route guards (not explicitly detailed but implied by architecture) govern dashboard access based on the reactive Signal role (Customer, Agent, ClaimOfficer, Admin).
3. **Data Communication:** Angular HTTP Interceptors sit between the Services and the Backend API, automatically attaching the raw JWT Bearer token into the `Authorization` headers for all secured HTTP requests. Responses return standard Observables which components subscribe (or via `toSignal`) to render UI asynchronously.
4. **Artifact Hydration:** The frontend retrieves direct external URLs for PDFs (from Vercel Blob) through Invoice dtos and renders them cleanly, decoupling large binaries from the local Angular bundle.

6.2 Frontend Service Functions

The frontend strictly delineates logic into singleton Services injected optimally at the root level (`providedIn: 'root'`).

- **AuthService :**
 - `register(dto: RegisterDto)` : Fires POST to `/api/auth/register` .
 - `login(dto: LoginDto)` : POSTs credentials, intercepts the resulting JWT, validates and decodes the payload, overrides localized state, and dumps it into `sessionStorage` .
 - `logout()` : Wipes JWT from `sessionStorage` and resets Signal state.
 - `isAuthenticated()` & `getRole()` : Helper pure functions assessing the real-time Signal state.
- **ClaimService :**
 - `submitClaim(dto)` : POSTs `/api/claims` wrapping multipart-forms.
 - `getMyClaims()` , `getAllClaims()` , `getAssignedClaims()` : Fetches reactive lists of claims.
 - `approveClaim(id, dto)` & `rejectClaim(id)` : Invokes officer mutations endpoints.
- **PolicyRequestService :**

- `createRequest(planId, duration, frequency, pan, address)` : Parses native `File` objects into a standard heavy `FormData` object and fires it to the backend `POST /api/policy-requests` .
- `getMyRequests()` , `getAssignedRequests()` : Fetch methods wrapping respective endpoints.
- `approveRequest(id)` , `rejectRequest(id, reason)` : Agent-sided approval pipelines.
- **PolicyService** :
 - `purchasePolicy(requestId)` : `POST /api/policies/purchase/{requestId}` . Transforms an approved request into a purchased artifact.
 - `getMyPolicies()` , `getPolicyById(id)` , `getAllPolicies()` , `getAssignedPolicies()` : Standard fetch pipelines routing the respective REST paths to Observables.
 - `payPolicy(paymentId)` : POSTs a payment initiation request against a schedule id.

(Note: Additional services like `NotificationService`, `PlanService`, and `ToastService` run identically, acting as thin, type-safe wrappers across the respective REST APIs)